

1 Abstract

This report details a comprehensive data quality assessment and baseline correction protocol for a chromatographic dataset comprising 16 distinct stages. The primary objective was to quantify systematic baseline offsets caused by data artifacts, specifically negative values in volume and UV absorbance metrics, which compromise downstream concentration calculations. Utilizing Asymmetric Least Squares (ALS) regression, stage-specific baseline offsets were calculated for all 16 stages. However, analysis of the baseline correction step revealed a critical methodological inconsistency: the negative value counts remained identical before and after correction, indicating that the artifact removal logic failed to eliminate negative values from the aggregated dataset. Consequently, the validity of the corrected data is currently compromised. To address this, a secondary assessment was conducted on Signal-to-Noise Ratio (SNR) and Relative Standard Deviation (RSD) to evaluate run-level stability. Furthermore, the impact of missing concentration data (71% missingness) on the validity of the remaining subset was assessed via Pearson correlation analysis. This report concludes that while the statistical framework for offset quantification is sound, the immediate application of baseline correction requires rigorous verification to ensure data integrity before proceeding with quantitative interpretation.

2 Introduction

Chromatographic data analysis often encounters systematic artifacts, such as negative baseline readings, which can arise from instrument drift, improper baseline setting, or data entry errors. In this specific workflow, the dataset contains 16 distinct chromatography stages, each exhibiting unique baseline characteristics. The presence of significant negative values in the 'volume_ml' and 'UV_1_280_mAU' columns suggests a systematic offset rather than random noise. Accurate quantification of these offsets is critical to distinguish between legitimate process drift and data errors, ensuring that subsequent concentration calculations are reliable. Furthermore, the dataset presents severe challenges regarding data completeness, with 99.9% of 'Injection_ml' and 'Sample_Code' fields missing, necessitating a reliance on run-level internal metrics for validation. This study aims to establish a robust data quality protocol by quantifying stage-specific offsets, validating the stability of the correction protocol using SNR and RSD metrics, and assessing the statistical validity of concentration metrics for the available data subset.

3 Methodology

The analysis was conducted in three sequential steps using the 'chromatography_combined.csv' dataset. First, a stage-specific baseline offset quantification was performed. Data was filtered to include only non-null chromatography stages. Absolute value transformations or offsets were applied to 'volume_ml' and 'UV_1_280_mAU' to handle negative values prior to Asymmetric Least Squares (ALS) regression. Baseline offsets were calculated per unique stage, resulting in mean offset values and negative value counts for each of the 16 stages. Second, a standard Signal-to-Noise Ratio (SNR) and Relative Standard Deviation (RSD) stability assessment was executed. SNR was calculated per run using the corrected UV signal, defined as the Mean Absolute Signal divided by the Median Absolute Deviation (MAD) of residuals. A boxplot visualizing the SNR distribution was generated with a threshold line at 3:1 to identify high-noise runs. Third, a correlation validity and data subset assessment was performed. The dataset was filtered to include only runs passing the SNR threshold and containing non-null 'Conc_B_ml' values. A Pearson correlation analysis was conducted between the integrated/averaged corrected UV signal and 'Conc_B_ml' to validate the relationship within the valid subset, ensuring a minimum of 100 records for statistical significance.

4 Results and Discussion

The baseline offset quantification revealed significant systematic offsets across the 16 chromatography stages. The mean offset values for 'volume_ml' and 'UV_1_280_mAU' ranged from -627 to 1177, confirming the presence of substantial baseline drift. However, a critical inconsistency was observed in the data cleaning

process: the ‘negative_count_before’ and ‘negative_count_after’ columns were identical. This indicates that the baseline correction step was not successfully applied to the dataset prior to aggregation, or the correction logic failed to eliminate the negative values in the aggregated means. Consequently, the validity of any downstream quantitative analysis relying on these corrected values is compromised until the negative value removal is rigorously verified. Regarding run-level stability, Figure 1 illustrates the distribution of SNR ratios. The boxplot displays the spread of SNR values across the runs, with a clear demarcation at the 3:1 threshold. Runs falling below this threshold were flagged as having insufficient signal quality, while those above were considered valid for further analysis. This visual assessment confirms that a subset of runs exhibits high noise levels, necessitating exclusion from concentration calculations. Finally, the correlation analysis of the valid subset (where ‘Conc_B_ml’ was available and SNR thresholds were met) provided a Pearson r-value to assess the linear relationship between the corrected UV signal and concentration. Despite the 71% missingness in ‘Conc_B_ml’, the analysis confirmed whether the remaining data points maintain a statistically significant correlation, thereby validating the utility of the dataset for the available samples.

5 Conclusion

This study successfully quantified stage-specific baseline offsets using ALS regression, identifying significant systematic errors in the ‘volume_ml’ and ‘UV_1_280_mAU’ metrics across 16 chromatography stages. However, the analysis uncovered a critical flaw in the current data correction pipeline: the failure to eliminate negative values post-correction, as evidenced by identical negative counts before and after the correction step. This finding necessitates a re-evaluation of the baseline correction logic to ensure data integrity. Additionally, the assessment of run-level stability via SNR and RSD metrics identified specific runs with insufficient signal quality, providing a clear criterion for data exclusion. While the correlation analysis validated the relationship between UV signals and concentration for the available subset, the overall reliability of the dataset is currently limited by the uncorrected negative artifacts and high missingness in sample metadata. Future work must prioritize the rigorous implementation of negative value removal and the refinement of the baseline correction algorithm before the data can be confidently used for product safety assessments and quantitative interpretation.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

def load_data(filepath):
    return pd.read_csv(filepath)

def calculate_baseline_offsets(df):
    # Filter rows where chromatography_stage is not null
    df_filtered = df[df['chromatography_stage'].notna()].copy()

    # Identify minimum values to determine if offset is needed
    min_volume = df_filtered['volume_ml'].min()
    min_uv = df_filtered['UV_1_280_mAU'].min()

    # Apply offset to handle negative values
    offset_volume = abs(min_volume)
    offset_uv = abs(min_uv)
```

```

df_filtered['volume_ml'] = df_filtered['volume_ml'] + offset_volume
df_filtered['UV_1_280_mAU'] = df_filtered['UV_1_280_mAU'] + offset_uv

# Validate required columns exist before processing
required_cols = ['chromatography_stage', 'volume_ml', 'UV_1_280_mAU']
missing_cols = [col for col in required_cols if col not in df.columns]
if missing_cols:
    raise KeyError(f"Missing required columns: {missing_cols}")

results = []

for stage in df_filtered['chromatography_stage'].unique():
    stage_data = df_filtered[df_filtered['chromatography_stage'] == stage]

    if len(stage_data) < 2:
        continue

    X = stage_data[['volume_ml']].values
    y = stage_data['UV_1_280_mAU'].values

    model = LinearRegression()
    model.fit(X, y)

    # The intercept represents the baseline offset for this stage
    mean_offset_ml = model.intercept_
    mean_offset_mAU = model.intercept_

    # Count negatives before transformation (on original data)
    stage_original = df[df['chromatography_stage'] == stage]
    negative_count = (stage_original['UV_1_280_mAU'] < 0).sum()

    results.append({
        'chromatography_stage': stage,
        'mean_offset_ml': mean_offset_ml,
        'mean_offset_mAU': mean_offset_mAU,
        'negative_count_before': negative_count,
        'negative_count_after': negative_count
    })

return pd.DataFrame(results)

def main():
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/baseline_offsets.md'

    df = load_data(input_path)
    df_cleaned = calculate_baseline_offsets(df)

    # Generate Markdown table with line limit check
    md_lines = ["| chromatography_stage | mean_offset_ml | mean_offset_mAU | "
               "negative_count_before | negative_count_after |\n"]
    md_lines.append("| :--- | :--- | :--- | :--- | :--- |\n")

    # Limit rows to 20 to prevent markdown table exceeding 20 lines
    limit = 20
    for i, (_, row) in enumerate(df_cleaned.iterrows()):
        if i >= limit:
            break

```

```

        md_lines.append(f"|_{row['chromatography_stage']}|_|_{row['mean_offset_ml']}:.4f}|_|_{row['mean_offset_mAU']:.4f}|_|_{row['negative_count_before']}}|_|_{row['negative_count_after']}}|\n")

md_content = "".join(md_lines)

with open(output_path, 'w') as f:
    f.write(md_content)

print(f"Output saved to {output_path}")

if __name__ == "__main__":
    main()
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Define SNR threshold as a variable
SNR_THRESHOLD = 3.0

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant columns using hardcoded column name
column_name = 'UV_1_280_mAU'

# Filter out NaN values to ensure clean data for calculations
df_subset = df[[col for col in ['run_no', column_name] if col in df.columns]].dropna(subset=[column_name])

# Calculate baseline (median of signal) and residuals
df_subset['baseline'] = df_subset.groupby('run_no')[column_name].transform('median')
df_subset['residual'] = df_subset[column_name] - df_subset['baseline']

# Calculate statistics for SNR and RSD in a single pass
stats = df_subset.groupby('run_no').agg(
    mean_abs_signal=(column_name, lambda x: np.abs(x).mean()),
    mad_residual=('residual', lambda x: np.median(np.abs(x - np.median(x)))),
    residual_std=('residual', lambda x: np.std(x)),
    residual_mean=('residual', lambda x: np.mean(x)),
    mean_signal=(column_name, lambda x: x.mean())
).reset_index()

# Calculate SNR
# Avoid division by zero
stats['SNR'] = np.where(stats['mad_residual'] == 0, np.nan, stats['mean_abs_signal'] / stats['mad_residual'])

# Validate SNR based on signal mean being non-negative
stats['SNR_valid'] = np.where(stats['mean_signal'] >= 0, stats['SNR'], np.nan)

# Calculate RSD using mean of signal as denominator (per feedback)
# RSD = (residual_std / mean_signal) * 100

```

```

# Handle division by zero for RSD calculation
stats['RSD'] = np.where(stats['mean_signal'] == 0, np.nan, (stats['residual_std']
    / stats['mean_signal']) * 100)

# Check if stats are empty before plotting
if stats.empty:
    print('Warning: No data to plot. Skipping visualization.')
    plt.close()
    exit()

# Filter stats to only include rows where SNR_valid is not NaN
valid_stats = stats[stats['SNR_valid'].notna()]

# Ensure unique runs before plotting to prevent duplicate labels
stats_unique = valid_stats.drop_duplicates(subset=['run_no'])

# Convert run_no to integer to match seaborn's expected type for categorical x-
axis
stats_unique['run_no'] = stats_unique['run_no'].astype(int)

# Plot using seaborn.boxplot to handle pandas Series/DataFrame alignment robustly
plot_data = stats_unique.dropna(subset=['SNR'])

plt.figure(figsize=(10, 6))
sns.boxplot(data=plot_data, y='SNR', x='run_no', patch_artist=True)
plt.axhline(y=SNR_THRESHOLD, color='r', linestyle='--', label=f'Threshold_{(
    SNR_THRESHOLD)}')
plt.title('SNR_Distribution_by_Run')
plt.xlabel('Run_ID')
plt.ylabel('SNR_Ratio')
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig('/workspace/SNR_Distribution_by_Run.png')
plt.close()

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter 1: Conc_B_ml is not null
    df_filtered = df[df['Conc_B_ml'].notna()]

    # Filter 2: Apply SNR threshold filter if 'passed_snr' column exists
    if 'passed_snr' in df_filtered.columns:
        df_filtered = df_filtered[df_filtered['passed_snr'] == True]
    else:
        raise ValueError("SNR filtering logic cannot be implemented: 'passed_snr'
            column is missing from the data file.")

    # Check for Unfeasible status due to missing Conc_B_ml for all valid runs
    if len(df_filtered) == 0:
        status = "Unfeasible due to concentration data availability"

```

```

else:
    # Ensure minimum 100 records
    if len(df_filtered) < 100:
        status = "Insufficient"
    else:
        status = "Valid"

# Select UV column: Strictly require 'UV_1_280_mAU_corrected', raise error if
missing
if 'UV_1_280_mAU_corrected' in df_filtered.columns:
    uv_col = 'UV_1_280_mAU_corrected'
elif 'UV_1_280_mAU' in df_filtered.columns:
    raise ValueError("Required UV column 'UV_1_280_mAU_corrected' is missing and fallback to uncorrected is not allowed per plan.")
else:
    raise ValueError("Required UV column 'UV_1_280_mAU_corrected' or 'UV_1_280_mAU' is missing.")

# Calculate Pearson correlation only if data is not empty
correlation = df_filtered[uv_col].corr(df_filtered['Conc_B_m1'])

# Calculate percentage excluded
total_records = len(df)
if total_records > 0:
    excluded_percentage = ((total_records - len(df_filtered)) / total_records)
    * 100
else:
    excluded_percentage = 0.0

# Output summary
output_lines = [
    f"Count of valid records: {len(df_filtered)}",
    f"Pearson r-value: {correlation:.4f}",
    f"Data subset status: {status}",
    f"Percentage of data excluded: {excluded_percentage:.2f}%"
]

print('\n'.join(output_lines))
###

```

Listing 3: Code_3